/Métodos/

Flora Belas

/Métodos/

- Um **método** é um bloco de código que executa uma ação ou retorna um valor, pertencente a uma classe (ou objeto). Ele é invocado pelo seu nome e pode receber parâmetros de entrada.

/Métodos/

- Serve pra não repetir e pra organizar
- Método e função são a mesma coisa em Java
- Nome de método é verbo: calcularMedia

/Parâmetros/

Parâmetro são as variáveis declarada entre os parênteses na definição do método. Ela "recebe" o valor que vai ser passado na chamada.

/Como criar um método/

- static void saudar() { System.out.println("Bem-vinda!"); }

Chamada sem parâmetros:
saudar();

static void saudarParametro(String nome) {
System.out.println("Bem-vinda, " + nome + "!");
}

Chamada com parâmetros:
saudarParametro(“Ana”);

/static/

static significa que o método **pertence à classe**, não a um objeto específico. Ou seja, você **não precisa criar uma instância** para chamá-lo.

Ao criarmos o Scanner para utilizar o nextInt, precisamos criar um objeto sc, e chamar sc.nextInt().
Um método static não precisa do sc (o novo objeto instanciado).

/return/

- **void** — declara que o método **não devolve nada**.
- **return** — envia um valor de volta para quem chamou.

Ao criar o método você precisa definir se ele retorna nada (void) ou o tipo de retorno (String, int, double...)

```
static int somar(){

return 0;
}
```

```
static int somar(int n1, int n2){
    int resultado = n1 + n2;
    return resultado;
}
```

```
static int somar(int n1, int n2){

return n1 + n2;
}
```

/Assinatura do Método/

- A **assinatura do método** é a combinação do nome do método e dos tipos, quantidade e ordem dos seus parâmetros, servindo para identificar o método de **forma única** dentro de uma classe. O tipo de retorno não faz parte da assinatura para fins de sobrecarga de método (overloading), significando que **dois métodos com o mesmo nome e parâmetros não podem existir na mesma classe, mesmo que retornem tipos diferentes.**

/Assinatura do Método/

No método:

```
static void saudacao(String nome) {
    System.out.println("Olá, " + nome + "!");
}
```

A assinatura é:

saudacao(String)

/Sobrecarga/

- Sobrecarga (sobrecarga de métodos) é a capacidade de ter múltiplos métodos com o mesmo nome na mesma classe, desde que **suas listas de parâmetros sejam diferentes**.

O que muda	Exemplo
Número de parâmetros	somar(int, int) vs somar(int, int, int)
Tipo dos parâmetros	somar(int, int) vs somar(double, double)
Ordem dos parâmetros	metodo(int, String) vs metodo(String, int)

/Exemplo de uso/

```
System.out.println("oi"); // println(String)
System.out.println(42); // println(int)
System.out.println(3.14); // println(double)
System.out.println(true); // println(boolean)
...
```

São **10 versões** de println na classe PrintStream, todas com o mesmo nome. Sem overloading você teria que lembrar **printlnString, printlnInt, printlnDouble**